

---

# DEEP LEARNING FOR IMAGE CLASSIFICATION

Corduneanu-Huci Mia  
Haltrich Thanika

2<sup>nd</sup> Year BA - DSS

---

# DATASETS:

CIFAR10 dataset

Fashion MNIST dataset

Required libraries:

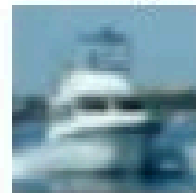
numpy, pandas, matplotlib.pyplot, torch, torchvision,  
tensorflow, sklearn, PIL, os, math, timexgboost,  
geodatasets, geopandas, math, shapely, random,  
tpot, flaml, inspect, time

---

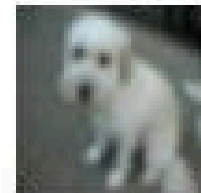
## DATASET 1

# CIFAR10

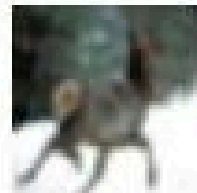
ship



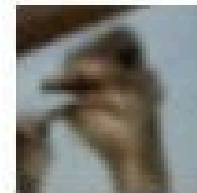
dog



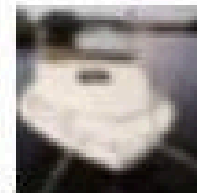
deer



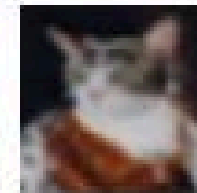
bird



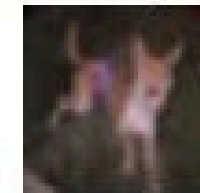
ship



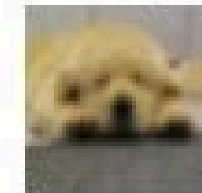
cat



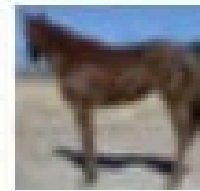
dog



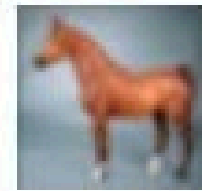
dog



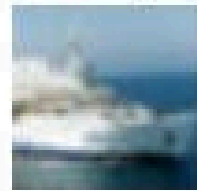
horse



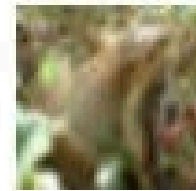
horse



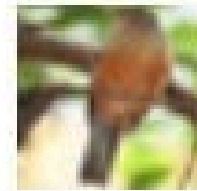
ship



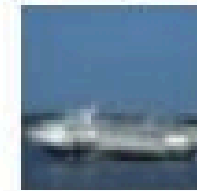
frog



bird



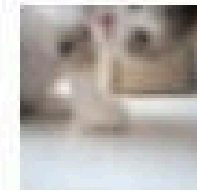
ship



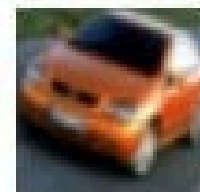
bird



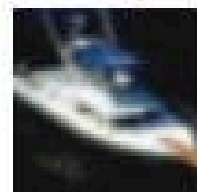
cat



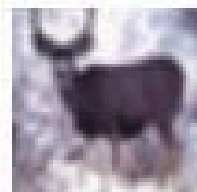
automobile



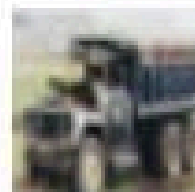
ship



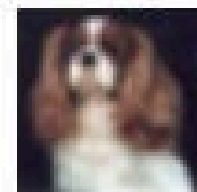
deer



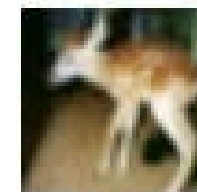
truck



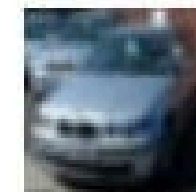
dog



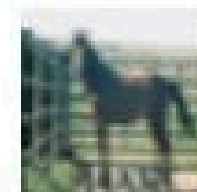
deer



automobile



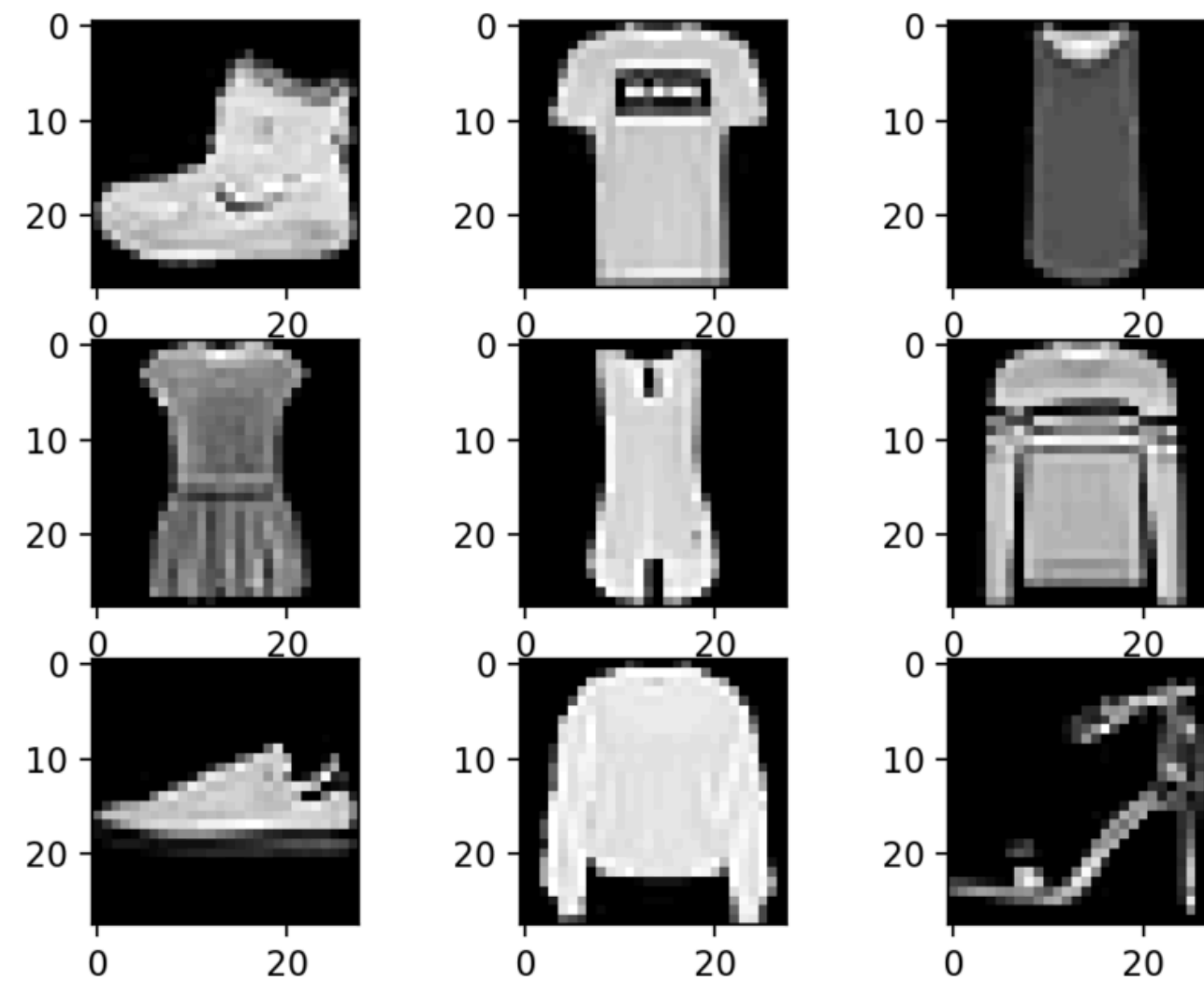
horse



---

## DATASET 2

# FASHION MNIST



---

# OUR COMPARISION

traditional and deep learning models

---

## TRADITIONAL AND DEEP LEARNING MODELS

### Traditional Models

- requires manual feature extraction
- we chose to run through Random Forest

### Deep Learning Models

- CNN
  - 3 different architectures
- ResNet 18 & 50
- Neural ODE

# RANDOM FOREST WITH FEATURE EXTRACTION

```
# — 2. Extract features —————

def extract_features(images):
    features = []
    for img in images:
        color_means = img.mean(axis=(0, 1))          # mean R, G, B → 3 dims
        color_stds  = img.std(axis=(0, 1))           # std  R, G, B → 3 dims
        gray        = img.mean(axis=2)               # grayscale → 32×32
        flat_gray   = gray.flatten()                 # pixel values → 1024 dims
        features.append(np.concatenate([color_means, color_stds, flat_gray]))
    return np.array(features)                        # total: 1030 dims

N = 10_000 # use a subset for speed; set to None for all 50k

print("Extracting features...")
X_train_feats = extract_features(X_train[:N])
X_test_feats  = extract_features(X_test[:2000])

# — 3. Train & evaluate —————

print("Training Random Forest...")
clf = RandomForestClassifier(n_estimators=100, n_jobs=-1, random_state=42)
clf.fit(X_train_feats, y_train[:N])

y_pred = clf.predict(X_test_feats)
print(classification_report(y_test[:2000], y_pred, target_names=[
    "airplane", "automobile", "bird", "cat", "deer",
    "dog", "frog", "horse", "ship", "truck",
]))
```

## Features to be used in RF:

- Average R, G, B brightness across the whole image
- How spread out / varied each colour channel is
- Raw grayscale pixel values, position by position

## Notes

- only using a subset to do computations

# RANDOM FOREST RESULTS

## DATASET 1

|              | precision | recall | f1-score |
|--------------|-----------|--------|----------|
| airplane     | 0.89      | 0.68   | 0.77     |
| automobile   | 0.99      | 0.91   | 0.95     |
| bird         | 0.89      | 0.61   | 0.72     |
| cat          | 0.92      | 0.77   | 0.84     |
| deer         | 0.82      | 0.59   | 0.69     |
| dog          | 0.97      | 0.85   | 0.91     |
| frog         | 0.86      | 0.41   | 0.55     |
| horse        | 0.92      | 0.89   | 0.90     |
| ship         | 0.99      | 0.89   | 0.94     |
| truck        | 0.93      | 0.90   | 0.92     |
| micro avg    | 0.92      | 0.75   | 0.83     |
| macro avg    | 0.92      | 0.75   | 0.82     |
| weighted avg | 0.92      | 0.75   | 0.82     |
| samples avg  | 0.75      | 0.75   | 0.75     |

## DATASET 2

|                  | precision | recall | f1-score | support |
|------------------|-----------|--------|----------|---------|
| T-shirt/top      | 0.80      | 0.81   | 0.80     | 200     |
| Trouser          | 0.97      | 0.93   | 0.95     | 203     |
| Pullover         | 0.70      | 0.78   | 0.73     | 214     |
| Dress            | 0.79      | 0.87   | 0.83     | 190     |
| Coat             | 0.73      | 0.75   | 0.74     | 219     |
| Sandal           | 0.90      | 0.88   | 0.89     | 195     |
| Shirt            | 0.63      | 0.50   | 0.56     | 197     |
| Sneaker          | 0.86      | 0.88   | 0.87     | 200     |
| Bag              | 0.95      | 0.92   | 0.93     | 194     |
| Ankle boot       | 0.90      | 0.91   | 0.90     | 188     |
| accuracy         |           |        | 0.82     | 2000    |
| macro avg        | 0.82      | 0.82   | 0.82     | 2000    |
| weighted avg     | 0.82      | 0.82   | 0.82     | 2000    |
| Accuracy: 0.8210 |           |        |          |         |

# 1ST CNN VARIATION

```
cnn_model = Sequential() # def a sequential CNN model
cnn_model.add(Conv2D(32, (3, 3), activation='relu', input_shape=(32, 32, 3))) # first layer, 32 convolutional neurons of size 3x3, relu activation
cnn_model.add(MaxPooling2D((2, 2))) # reduces spatial dimensions
cnn_model.add(Dropout(0.25)) # reduce overfitting (randomly sets 25% of inputs to 0 during training)
cnn_model.add(Flatten()) # flatten the 2D feature maps into a 1D vector for fully connected vectors
cnn_model.add(Dense(128, activation='relu')) # fully connected layer
cnn_model.add(Dropout(0.5)) # 50% of neurons dropped
cnn_model.add(Dense(10, activation='softmax')) # output layer -> 10 neurons (softmax activation for probabilities)
# dense fully connected layer(with 256)
cnn_model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy']) # compiles the model
cnn_history = cnn1_start_train = time.time()
cnn_model.fit(X_train, y_train, epochs=12, batch_size=32, validation_data=(X_test, y_test))
cnn1_train_time = time.time() - cnn1_start_train
# Test timing
cnn1_start_test = time.time()
_ = cnn_model.predict(X_test, verbose=0)
cnn1_test_time = time.time() - cnn1_start_test
_, cnn1_acc = cnn_model.evaluate(X_test, y_test, verbose=0)
print(f"Accuracy: {cnn1_acc:.4f} | Train: {cnn1_train_time:.1f}s | Test: {cnn1_test_time:.2f}s") # trains the model on the training set
```

---

## 2ND CNN VARIATION

```
cnn_model_2 = Sequential()
cnn_model_2.add(Input(shape=(32,32,3)))

# more convolutional layers
# first layer
cnn_model_2.add(Conv2D(16, (3, 3), activation='relu'))
cnn_model_2.add(MaxPooling2D((2, 2)))

# second layer
cnn_model_2.add(Conv2D(32, kernel_size=(3,3), activation='relu'))
cnn_model_2.add(MaxPooling2D(pool_size=(2,2)))

cnn_model_2.add(Flatten())
cnn_model_2.add(Dense(128, activation='relu'))
cnn_model_2.add(Dropout(0.5))
cnn_model_2.add(Dense(10, activation='softmax'))

cnn_model_2.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
cnn_history = cnn2_start_train = time.time()
cnn_model_2.fit(X_train, y_train, epochs=12, batch_size=32, validation_data=(X_test, y_test))
cnn2_train_time = time.time() - cnn2_start_train
# Test timing
cnn2_start_test = time.time()
_ = cnn_model_2.predict(X_test, verbose=0)
cnn2_test_time = time.time() - cnn2_start_test
_, cnn2_acc = cnn_model_2.evaluate(X_test, y_test, verbose=0)
print(f"Accuracy: {cnn2_acc:.4f} | Train: {cnn2_train_time:.1f}s | Test: {cnn2_test_time:.2f}s")
```

---

## 3RD CNN VARIATION

```
cnn_model_3 = Sequential()
cnn_model_3.add(Input(shape=(32,32,3)))

# first layer
cnn_model_3.add(Conv2D(32, (3, 3), activation='relu'))
cnn_model_3.add(MaxPooling2D((2, 2)))

# second layer
cnn_model_3.add(Conv2D(64, (3, 3), activation='relu'))
cnn_model_3.add(MaxPooling2D((2, 2)))

# third layer
cnn_model_3.add(Conv2D(128, (3, 3), activation='relu'))
#cnn_model_3.add(MaxPooling2D((2,2)))

cnn_model_3.add(Flatten())
cnn_model_3.add(Dense(128, activation='relu'))
cnn_model_3.add(Dropout(0.5))
cnn_model_3.add(Dense(10, activation='softmax'))

cnn_model_3.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
cnn_history = cnn3_start_train = time.time()
cnn_model_3.fit(X_train, y_train, epochs=12, batch_size=32, validation_data=(X_test, y_test))
cnn3_train_time = time.time() - cnn3_start_train
# Test timing
cnn3_start_test = time.time()
_ = cnn_model_3.predict(X_test, verbose=0)
cnn3_test_time = time.time() - cnn3_start_test
_, cnn3_acc = cnn_model_3.evaluate(X_test, y_test, verbose=0)
print(f"Accuracy: {cnn3_acc:.4f} | Train: {cnn3_train_time:.1f}s | Test: {cnn3_test_time:.2f}s")
```

---

## CNN WITH AVG. POOLING

```
cnn_model_4 = Sequential()
cnn_model_4.add(Input(shape=(28,28,1)))

# first layer
cnn_model_4.add(Conv2D(32, (3, 3), activation='relu'))
cnn_model_4.add(AveragePooling2D((2,2))) # reduces the spatial dimensions

# second layer
cnn_model_4.add(Conv2D(64, (3, 3), activation='relu'))
cnn_model_4.add(AveragePooling2D((2,2)))

# third layer
cnn_model_4.add(Conv2D(128, (3, 3), activation='relu'))
cnn_model_4.add(AveragePooling2D((2,2)))

cnn_model_4.add(Flatten()) # 3D feature maps into 1D vector
cnn_model_4.add(Dense(128, activation='relu'))
cnn_model_4.add(Dropout(0.5))
cnn_model_4.add(Dense(10, activation='softmax'))
```

---

# CNN WITH BATCH NORMALIZATION

```
# Block 1
cnn_extra.add(Conv2D(32, (3, 3), activation='relu', padding='same'))
cnn_extra.add(BatchNormalization())
cnn_extra.add(Conv2D(32, (3, 3), activation='relu', padding='same'))
cnn_extra.add(BatchNormalization())
cnn_extra.add(MaxPooling2D((2, 2)))
cnn_extra.add(Dropout(0.2))

# Block 2
cnn_extra.add(Conv2D(64, (3, 3), activation='relu', padding='same'))
cnn_extra.add(BatchNormalization())
cnn_extra.add(Conv2D(64, (3, 3), activation='relu', padding='same'))
cnn_extra.add(BatchNormalization())
cnn_extra.add(MaxPooling2D((2, 2)))
cnn_extra.add(Dropout(0.3))

# Block 3
cnn_extra.add(Conv2D(128, (3, 3), activation='relu', padding='same'))
cnn_extra.add(BatchNormalization())
cnn_extra.add(MaxPooling2D((2, 2)))
cnn_extra.add(Dropout(0.4))

cnn_extra.add(Flatten())
cnn_extra.add(Dense(256, activation='relu'))
cnn_extra.add(BatchNormalization())
cnn_extra.add(Dropout(0.5))
cnn_extra.add(Dense(10, activation='softmax'))
```

---

## CNN WITH EXTENSIVE DROPOUT

```
cnn_model_d = Sequential()
cnn_model_d.add(Input(shape=(28,28,1)))

# first layer
cnn_model_d.add(Conv2D(32, (3, 3), activation='relu'))
cnn_model_d.add(Dropout(0.25)) # reduces overfitting (25% of neurons dropped)
cnn_model_d.add(MaxPooling2D((2, 2)))

# second layer
cnn_model_d.add(Conv2D(64, (3, 3), activation='relu'))
cnn_model_d.add(Dropout(0.25)) # reduces overfitting (25% of neurons dropped)
cnn_model_d.add(MaxPooling2D((2, 2)))

# third layer
cnn_model_d.add(Conv2D(128, (3, 3), activation='relu'))

cnn_model_d.add(Flatten())
cnn_model_d.add(Dense(128, activation='relu'))
cnn_model_d.add(Dropout(0.5)) # 50% neurons dropped
cnn_model_d.add(Dense(10, activation='softmax'))
```

|                     | CIFAR10 | Fashion MNSIT |
|---------------------|---------|---------------|
| CNN1                | 0.6284  | 0.8843        |
| CNN2                | 0.6634  | 0.8825        |
| CNN3                | 0.7165  | 0.9072        |
| AVG. POOLING        | 0.7190  | 0.8813        |
| Batch. Norm.        | 0.6920  | 0.8968        |
| Drop out            | 0.7447  | 0.8968        |
| Data Augmentation   | 0.6920  | 0.7331        |
| Hyper param. tuning | 0.7110  | 0.8800        |

---

## RESNET50

```
# ResNet50 expects at least 32x32 but was designed for larger images
# We use include_top=False to remove the original classification head
base_model = ResNet50(weights=None, # train from scratch (no pretrained weights)
                      include_top=False,
                      input_shape=(32, 32, 3))
```

```
resnet_model.compile(optimizer='adam',
                    loss='categorical_crossentropy',
                    metrics=['accuracy'])
```

# RESNET18

```
def resnet18(num_classes):  
    """Constructs a ResNet-18 model."""  
    return ResNet(block=BasicBlock,  
                  layers=[2, 2, 2, 2], # makes it a ResNet18  
                  num_classes=num_classes,  
                  grayscale=GRAYSCALE)
```

|          | CIFAR10 | Fashion MNSIT |
|----------|---------|---------------|
| ResNet50 | 0.3433  | 0.8700        |
| ResNet18 | 0.7544  | 0.7274        |

## N-ODE

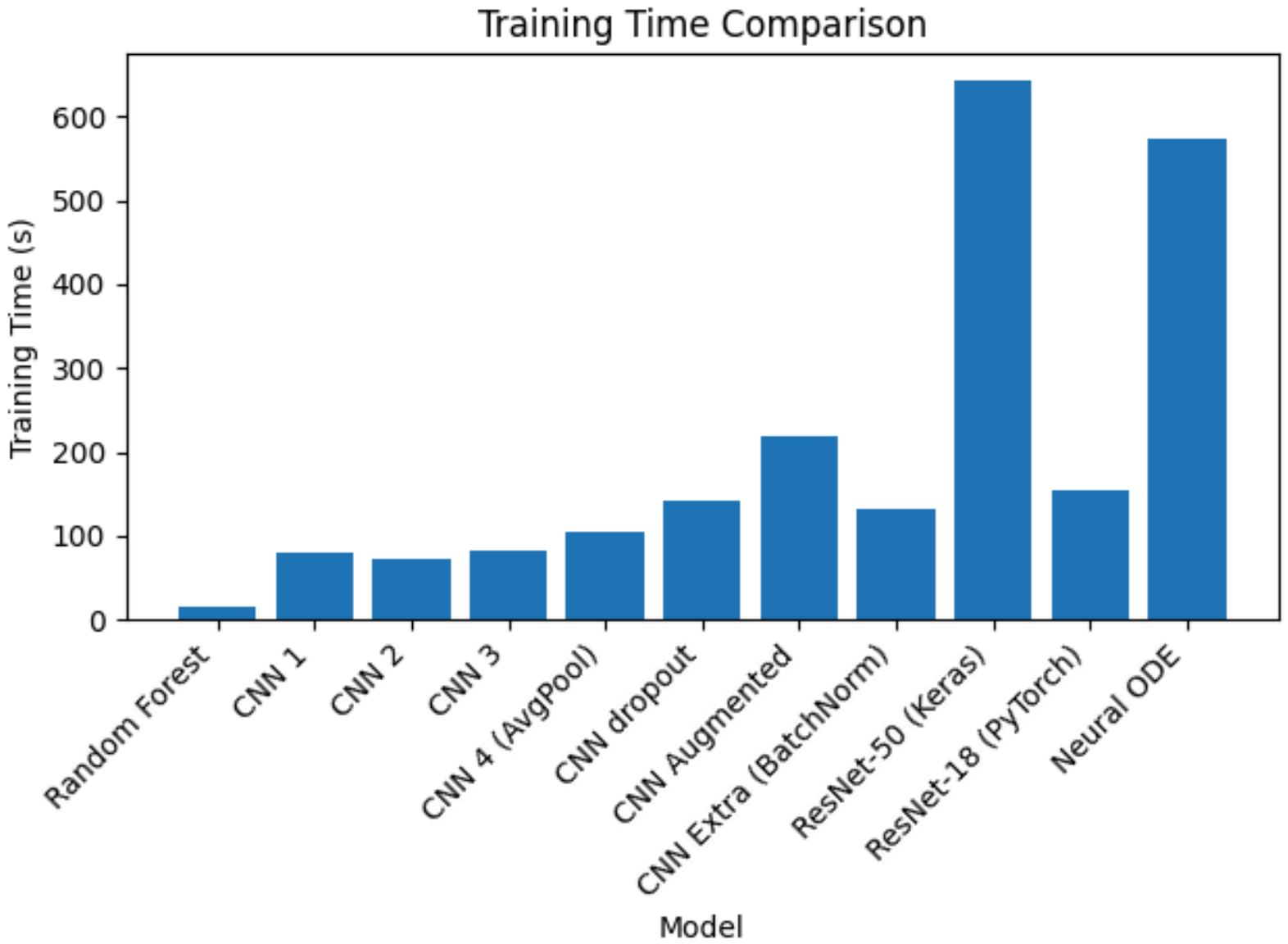
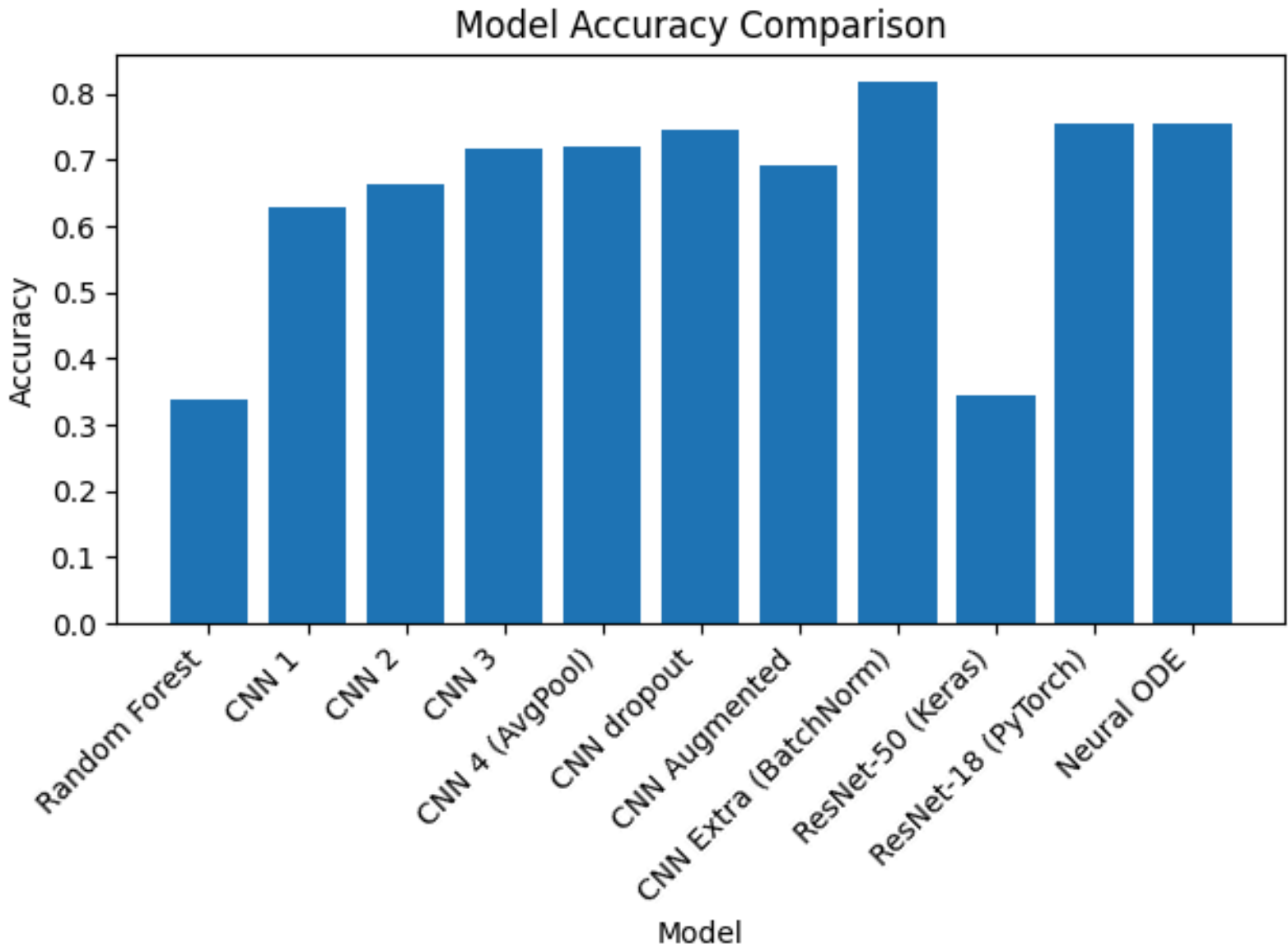
|            | CIFAR10 | Fashion MNSIT |
|------------|---------|---------------|
| Neural ODE | 0.7544  | 0.9164        |

```
class ODEAdjoint(torch.autograd.Function):
    @staticmethod
    def forward(ctx, z0, t, flat_parameters, func):
        '''calling the ode solver for each time step'''
        assert isinstance(func, ODEF)
        bs, *z_shape = z0.size()
        time_len = t.size(0)

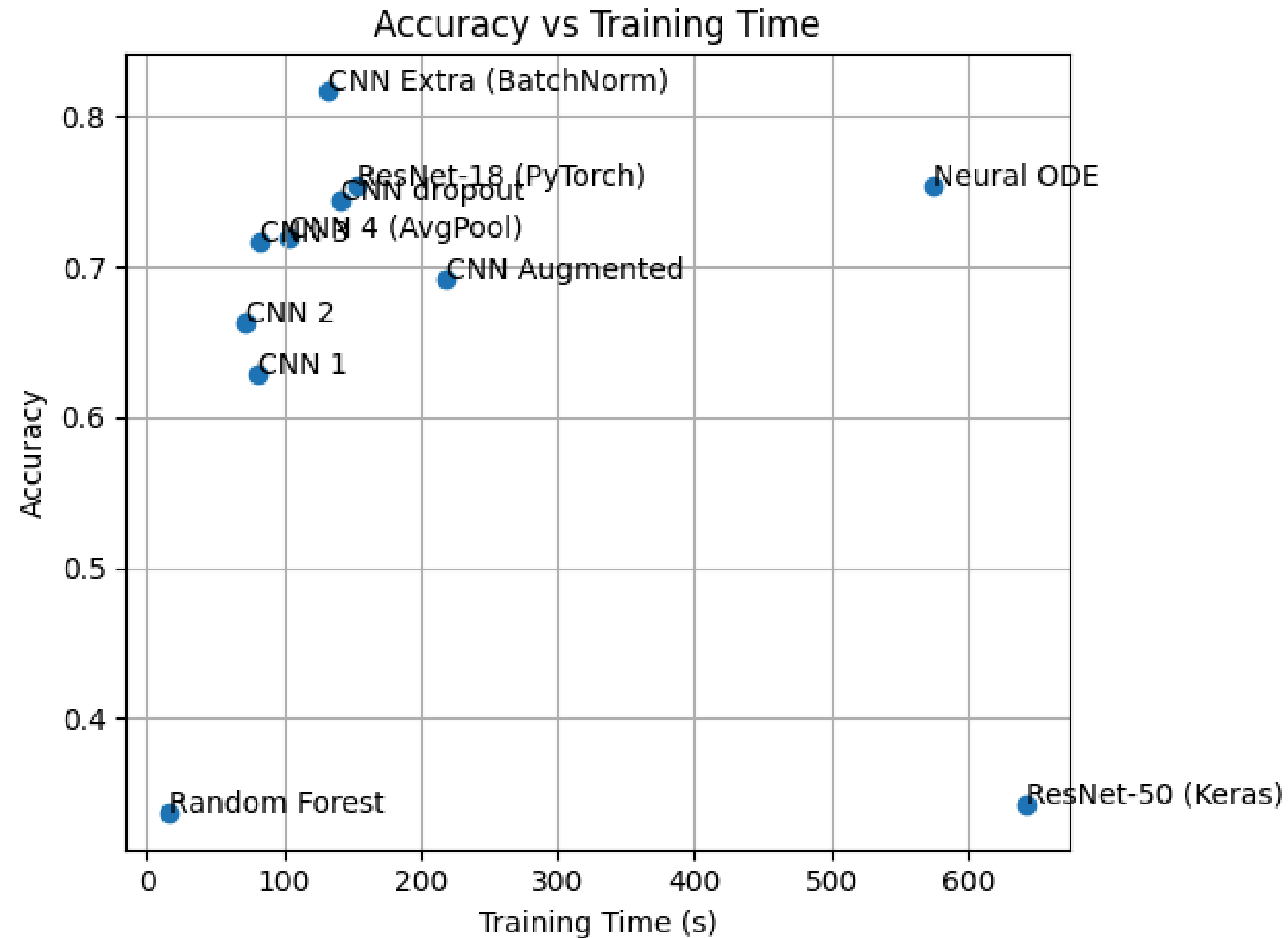
        with torch.no_grad(): #we don't save gradients
            z = torch.zeros(time_len, bs, *z_shape).to(z0)
            z[0] = z0
            for i_t in range(time_len - 1): # --(1)
                z0 = ode_solve(z0, t[i_t], t[i_t+1], func)
                z[i_t+1] = z0

        ctx.func = func
        ctx.save_for_backward(t, z.clone(), flat_parameters)
        #ctx is what is saved and will be used for backprop later
        return z
```

# CIFRA-10



# CIFRA-10



---

**THANK YOU!**